

Linux Commands

Class no:11

1.FILE PERMISSIONS:

- Definition: File permissions specify the allowed actions (read, write, execute) for different categories of users on a file or directory.
- Purpose: They prevent unauthorized access, protect sensitive data, and maintain system stability.
- Categories of Users:
 - Owner (u) → The user who created or owns the file.
 - Group (g) → Users in the file's group.
 - Others (o) → All other users on the system.

Types of Permissions

Permission	Symbol	Numeric Value	Meaning
Read	r	4	View file contents / list directory
Write	w	2	Modify file / add or remove files in directory
Execute	x	1	Run file as program / enter directory

Numeric Representation (chmod)

- Permissions are expressed as three digits, one for each user category (owner, group, others).
- Each digit is the sum of the values for read (4), write (2), and execute (1).

Examples:

- chmod 755 file.txt → Owner: rwx (7), Group: r-x (5), Others: r-x (5)
- chmod 644 file.txt → Owner: rw- (6), Group: r-- (4), Others: r-- (4)
- chmod 700 file.txt → Owner: rwx (7), Group: --- (0), Others: --- (0)

Symbolic Representation

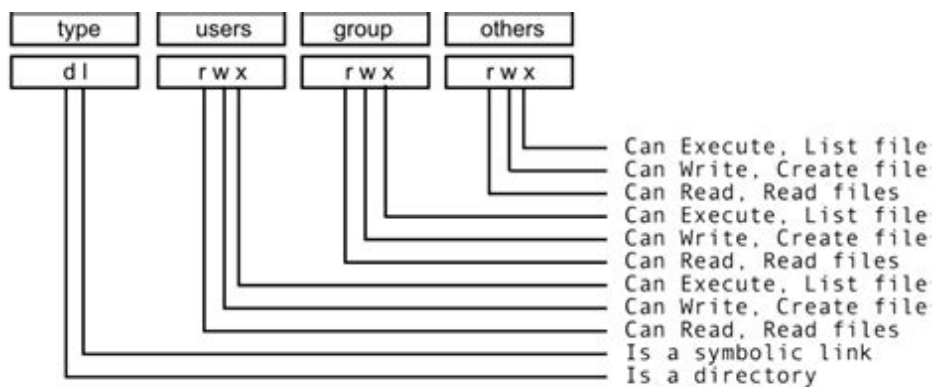
Instead of numbers, you can use letters:

- chmod u+x file → Add execute for owner
- chmod g-w file → Remove write for group
- chmod o=r file → Set others to read only

Why Permissions Matter

- Security: Prevents unauthorized users from reading or altering sensitive files.
- System Integrity: Stops accidental or malicious execution of critical files.
- Collaboration: Allows controlled sharing of files among groups.

Linux File Permissions



Hands on: Visualising file permissions and changing it.

Step 1: First create a directory (*folder1*) from the home and move inside of it using cd absolute path. Create a subfolder (*subfolder1*) and two more files (*fileno1.txt* & *fileno2.txt*).

```
[ec2-user@ip-172-31-33-212 ~]$ mkdir folder1
[ec2-user@ip-172-31-33-212 ~]$ cd folder1
[ec2-user@ip-172-31-33-212 folder1]$ mkdir subfolder1
[ec2-user@ip-172-31-33-212 folder1]$ touch fileno1.txt fileno2.txt
```

ls -l -> longlisting file information with timestamp, file permissions and ownership.

```
[ec2-user@ip-172-31-33-212 ~]$ cd folder1
[ec2-user@ip-172-31-33-212 folder1]$ ls -l
total 0
-rw-r--r--. 1 ec2-user ec2-user 0 Mar 23 13:20 fileno1.txt
-rw-r--r--. 1 ec2-user ec2-user 0 Mar 23 13:20 fileno2.txt
drwxr-xr-x. 2 ec2-user ec2-user 6 Mar 23 13:20 subfolder1
[ec2-user@ip-172-31-33-212 folder1]$
```

Step 2: To change permissions of file/folder : **chmod 777 filename**

777 -> defines the permissions for user, groups and others can perform read(r), write(w) and execute(x).

```
drwxr-xr-x. 2 ec2-user ec2-user 6 Mar 23 13:20 subfolder1
[ec2-user@ip-172-31-33-212 folder1]$ chmod 777 subfolder1
[ec2-user@ip-172-31-33-212 folder1]$ ls -l
total 0
-rw-r--r--. 1 ec2-user ec2-user 0 Mar 23 13:20 fileno1.txt
-rw-r--r--. 1 ec2-user ec2-user 0 Mar 23 13:20 fileno2.txt
drwxrwxrwx. 2 ec2-user ec2-user 6 Mar 23 13:20 subfolder1
```

We can see all the permissions has been enabled for the subfolder1 directory.

2.PACKAGE MANAGERS:

Package management is the process of installing, updating, configuring, and removing software packages on a system. A *package* typically contains compiled program files, metadata (name, version, dependencies), and configuration scripts.

A package manager automates these tasks, ensuring smooth software administration by:

- Resolving dependencies automatically.
- Fetching packages from trusted repositories.
- Keeping software updated to the latest secure versions.
- Removing packages cleanly without leaving orphaned files.

Different Linux distributions use different package managers, such as

- APT (Debian/Ubuntu also for *ubuntu ec2*) -> Advanced packaging tool
- DNF/YUM (Fedora/CentOS/RHEL also *amazon Linux ec2*)
The packages which cannot be installed by yum, can be installed by dnf.
- Pacman (Arch), and
- Zypper (openSUSE).

Example Commands (APT – Debian/Ubuntu-based systems).

Hands on for installing packages using package manager:

Step no 1: As we are using amazon Linux ec2, the package manager to install is yum. Note that we must have root user access. So add sudo in prefix.

yum install packagename (e.g. yum install tree)

```
[ec2-user@ip-172-31-33-212 folder1]$ sudo yum install tree
Total download size: 56 k
Installed size: 113 k
Is this ok [y/N]: y
Downloading Packages:
tree-1.8.0-6.amzn2023.0.2.x86_64.rpm
```

Provide y for yes. After installing it start the package using systemctl command.

3.FILE OWNERSHIP:

- Owner (User): The person who created the file by default. They have primary control over its permissions.
- Group: A collection of users. Files can be assigned to a group so multiple users share access.
- Others: All remaining users who are neither the owner nor part of the group.

File Permissions

Permissions determine what actions can be performed:

- Read (r): View the file's contents.
- Write (w): Modify or delete the file.
- Execute (x): Run the file as a program/script.

These permissions are applied separately to:

- Owner
- Group
- Others

Example: -rw-r--r--

- Owner: read & write
- Group: read only
- Others: read only

Importance of File Ownership

- Security: Prevents unauthorized access or modification of files.
- Collaboration: Groups allow shared access among team members.
- System Integrity: Protects critical system files from accidental or malicious changes.

Key Notes

- Every file in Linux has exactly one owner and one group.
- Ownership can be changed only by the root user or the current owner.
- Permissions are the first line of defence in Linux security.
- Proper ownership setup is essential in multi-user systems like servers.

Hands on to change ownership for a file:

Step no 1: To change the file's owner.

chown user file (ex. chown root fileno1.txt)

```
-rw-r--r--. 1 ec2-user ec2-user 0 Mar 23 13:20 fileno1.txt
-rw-r--r--. 1 ec2-user ec2-user 0 Mar 23 13:20 fileno2.txt
drwxrwxrwx. 2 ec2-user ec2-user 6 Mar 23 13:20 subfolder1
[root@ip-172-31-33-212 folder1]# chown root fileno1.txt
[root@ip-172-31-33-212 folder1]# ls -l
total 0
-rw-r--r--. 1 root ec2-user 0 Mar 23 13:20 fileno1.txt
-rw-r--r--. 1 ec2-user ec2-user 0 Mar 23 13:20 fileno2.txt
drwxrwxrwx. 2 ec2-user ec2-user 6 Mar 23 13:20 subfolder1
```

Step no 2: To change the group.

chown :group filename (ex. chown :root fileno1.txt)

```
-rw-r--r--. 1 root ec2-user 0 Mar 23 13:20 fileno1.txt
-rw-r--r--. 1 ec2-user ec2-user 0 Mar 23 13:20 fileno2.txt
drwxrwxrwx. 2 ec2-user ec2-user 6 Mar 23 13:20 subfolder1
[root@ip-172-31-33-212 folder1]# chown :root fileno1.txt
[root@ip-172-31-33-212 folder1]# ls -l
total 0
-rw-r--r--. 1 root root 0 Mar 23 13:20 fileno1.txt
-rw-r--r--. 1 ec2-user ec2-user 0 Mar 23 13:20 fileno2.txt
drwxrwxrwx. 2 ec2-user ec2-user 6 Mar 23 13:20 subfolder1
```

Step no 3: To change both owner and group.

chown user:group file (ex. chown ec2-user:ec2-user fileno1.txt)

```
-rw-r--r--. 1 root root 0 Mar 23 13:20 fileno1.txt
-rw-r--r--. 1 ec2-user ec2-user 0 Mar 23 13:20 fileno2.txt
drwxrwxrwx. 2 ec2-user ec2-user 6 Mar 23 13:20 subfolder1
[root@ip-172-31-33-212 folder1]# chown ec2-user:ec2-user fileno1.txt
[root@ip-172-31-33-212 folder1]# ls -l
total 0
-rw-r--r--. 1 ec2-user ec2-user 0 Mar 23 13:20 fileno1.txt
-rw-r--r--. 1 ec2-user ec2-user 0 Mar 23 13:20 fileno2.txt
drwxrwxrwx. 2 ec2-user ec2-user 6 Mar 23 13:20 subfolder1
```

To add user: **adduser username**

Ex. adduser nithya.

We can see the available users under /home folder.

```
[root@ip-172-31-33-212 folder1]# adduser nithya
[root@ip-172-31-33-212 folder1]# cd /home
[root@ip-172-31-33-212 home]# ls
ec2-user nithya
```

4.NETWORKING COMMENTS:

In Linux, when people talk about networking commands (sometimes mistakenly called "comments"), they mean the set of tools used to test, configure, and troubleshoot network connectivity. These commands help you interact with network interfaces, check connectivity, resolve domain names, and transfer data.

➤ **ifconfig**

- Stands for *interface configuration*.
- Used to display and configure network interfaces (public and private Ip address)
- Shows IP address, MAC address, netmask, and status of interfaces.

```
[root@ip-172-31-33-212 home]# ifconfig
ens5: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>
    inet 172.31.33.212 netmask 255.255.240.0
    inet6 fe80::8a0:9cff:fe5e:b6b9 prefixlen 64 scopeid 0x20:::
    ether 0a:a0:9c:5e:b6:b9 txqueuelen 1000
    RX packets 47069 bytes 62364520 (59.4 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 6115 bytes 631454 (616.6 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0
    collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 12 bytes 1020 (1020.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 12 bytes 1020 (1020.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0
    collisions 0

[root@ip-172-31-33-212 home]#
```

i-09c14a459a8fa7cf2 (My linux machine)

PublicIPs: 13.48.57.139 PrivateIPs: 172.31.33.212

➤ **Ping**

- Tests connectivity between two hosts.
- Sends ICMP echo requests and measures response time.
- Useful for checking if a server or website is reachable.
- Example: **ping private_ip_address_of_the_instance**

```
[root@ip-172-31-33-212 home]# ping 172.31.33.212
PING 172.31.33.212 (172.31.33.212) 56(84) bytes of data.
64 bytes from 172.31.33.212: icmp_seq=1 ttl=127 time=0.022 ms
64 bytes from 172.31.33.212: icmp_seq=2 ttl=127 time=0.032 ms
64 bytes from 172.31.33.212: icmp_seq=3 ttl=127 time=0.032 ms
64 bytes from 172.31.33.212: icmp_seq=4 ttl=127 time=0.026 ms
64 bytes from 172.31.33.212: icmp_seq=5 ttl=127 time=0.027 ms
64 bytes from 172.31.33.212: icmp_seq=6 ttl=127 time=0.032 ms
64 bytes from 172.31.33.212: icmp_seq=7 ttl=127 time=0.029 ms
```

➤ wget

- Command-line utility for downloading files from the web.
- Supports HTTP, HTTPS, and FTP.
- Can resume interrupted downloads.
- Example: **wget https://example.com/file.zip**

```
[ec2-user@ip-172-31-33-212 ~]$ wget https://example.com/file.zip.  
--2026-03-23 14:38:55-- https://example.com/file.zip.  
Resolving example.com (example.com)... 104.18.26.120, 104.18.27.120, 2606:4700::6812:1b78, ..  
Connecting to example.com (example.com)|104.18.26.120|:443... connected.  
ERROR: The certificate of 'example.com' is not trusted.  
ERROR: The certificate of 'example.com' doesn't have a known issuer.
```

➤ curl

- Transfers data to or from a server using various protocols (HTTP, HTTPS, FTP, etc.).
- More versatile than wget for APIs and testing endpoints.
- Example: **curl -I https://example.com** → fetches HTTP headers.

```
[ec2-user@ip-172-31-33-212 ~]$ curl -I https://example.com  
curl: (60) SSL certificate OpenSSL verify result: unable to  
More details here: https://curl.se/docs/sslcerts.html
```

➤ telnet

- Used to connect to remote machines on a specific port.
- Helps test connectivity to services (like SMTP on port 25).
- Note: Insecure (unencrypted), mostly replaced by SSH.

First install telnet package using yum command.

```
[root@ip-172-31-33-212 ~]# sudo yum install telnet  
Last metadata expiration check: 0:45:08 ago on Mon Mar 23 13:48:59 2026.  
Dependencies resolved.
```

- Example: **telnet public_ip 22** (ssh port).

```
Complete!  
[root@ip-172-31-33-212 ~]# telnet 13.48.57.139 22  
Trying 13.48.57.139...  
Connected to 13.48.57.139.  
Escape character is '^['.  
SSH-2.0-OpenSSH 8.7
```


➤ nslookup

- Queries DNS servers to resolve domain names into IP addresses. It shows the do
- Useful for troubleshooting DNS issues.
- Example: **nslookup public_ip_of_the_ec2_instance.**

```
root@ip-172-31-33-212 ~# nslookup 13.48.57.139
139.57.48.13.in-addr.arpa      name = ec2-13-48-57-139.eu-north-1.compute.amazonaws.com.
```

5.DISK MANAGEMENT COMMANDS:

Disk management commands in Linux are used to monitor disk usage, create and manage partitions, format disks, mount/unmount file systems, and check/repair storage. The most important commands include df, du, fdisk, cfdisk, lsblk, mkfs, mount/umount, fsck, and dd.

Key Disk Management Commands in Linux

1. **df** (disk free)

- Displays disk space usage of mounted file systems.
- Options:
 - **df -h** → human-readable format (GB/MB).

```
[ec2-user@ip-172-31-33-212 ~]$ df -h
Filesystem      Size  Used Avail Use% Mounted on
devtmpfs        4.0M   0    4.0M   0% /dev
tmpfs           459M   0    459M   0% /dev/shm
tmpfs           184M  432K   183M   1% /run
/dev/nvme0n1p1  8.0G  1.7G   6.4G  21% /
tmpfs           459M   0    459M   0% /tmp
/dev/nvme0n1p128 10M   1.3M   8.7M  13% /boot/efi
tmpfs           92M   0     92M   0% /run/user/1000
```

- **df -T** → shows filesystem type.

```
[ec2-user@ip-172-31-33-212 ~]$ df -T
Filesystem      Type      1K-blocks    Used Available Use% Mounted on
devtmpfs        devtmpfs      4096          0      4096     0% /dev
tmpfs           tmpfs        469420         0    469420     0% /dev/shm
tmpfs           tmpfs        187768         432   187336     1% /run
/dev/nvme0n1p1  xfs         8310764 1679576   6631188   21% /
tmpfs           tmpfs        469420         0    469420     0% /tmp
/dev/nvme0n1p128 vfat         10202        1318     8884    13% /boot/efi
tmpfs           tmpfs        93884          0    93884     0% /run/user/1000
```


2. **du** (disk usage)

- Shows disk space used by files/directories.

```
[ec2-user@ip-172-31-33-212 ~]$ du -h
4.0K    ./ssh
0       ./folder1/subfolder1
0       ./folder1
```

3. **lsblk**

- Lists block devices (disks and partitions).
- Shows hierarchy of devices.

```
[ec2-user@ip-172-31-33-212 ~]$ lsblk
NAME                MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
nvme0n1              259:0    0   8G  0 disk
├─nvme0n1p1          259:1    0   8G  0 part /
├─nvme0n1p127        259:2    0   1M  0 part
└─nvme0n1p128        259:3    0  10M  0 part /boot/efi
```

6.PROCESS COMMAND

In Linux, **process commands** are used to monitor, control, and manage running programs (processes). A *process* is simply an instance of a program in execution. These commands help administrators and users check system activity, terminate unwanted tasks, and optimize performance.

Important Process Commands in Linux

- ps (process status)** : Displays information about active processes.

Example:

- ps** → shows processes in the current shell.

```
[ec2-user@ip-172-31-33-212 ~]$ ps
  PID TTY          TIME CMD
 30501 pts/0    00:00:00 bash
 30951 pts/0    00:00:00 ps
```

- ps aux** → shows all processes with detailed info.

```
[ec2-user@ip-172-31-33-212 ~]$ ps aux
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  1.8 172556 17388 ?        Ss   12:40   0:01 /usr/lib/systemd/systemd
root         2  0.0  0.0      0     0 ?        S    12:40   0:00 [kthreadd]
root         3  0.0  0.0      0     0 ?        I<   12:40   0:00 [rcu_gp]
root         4  0.0  0.0      0     0 ?        I<   12:40   0:00 [rcu_par_gp]
root         5  0.0  0.0      0     0 ?        I<   12:40   0:00 [slub_flushwq]
root         6  0.0  0.0      0     0 ?        I<   12:40   0:00 [netns]
root         8  0.0  0.0      0     0 ?        I<   12:40   0:00 [kworker/0:0H-events_high]
root        10  0.0  0.0      0     0 ?        I<   12:40   0:00 [mm_percpu_wq]
root        11  0.0  0.0      0     0 ?        I    12:40   0:00 [rcu_tasks_kthread]
```

2. top

- Real-time view of running processes, CPU, and memory usage.
- Interactive: you can sort, kill processes, etc.

Example: **top**.

```
[ec2-user@ip-172-31-33-212 ~]$ top
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
826	root	20	0	53624	15860	14688	S	0.3	1.7	0:00.53	systemd-journal
1602	chrony	20	0	86460	3980	3296	S	0.3	0.4	0:00.17	chronyd
1	root	20	0	172556	17388	10792	S	0.0	1.9	0:01.57	systemd
2	root	20	0	0	0	0	S	0.0	0.0	0:00.00	kthreadd
3	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	rcu_gp
4	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	rcu_par_gp
5	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	slub_flushwq
6	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	netns

3. htop (enhanced top)

- Interactive process viewer (colorful, user-friendly).
- Allows scrolling and searching.

Example: **htop**

```
ec2-user@ip-172-31-33-212 ~]$ htop
```

4. systemctl

- Manages system services and processes (on systemd-based systems).

Example:

- **systemctl start httpd** → To start a service.
- **systemctl status httpd** → check service status.

```
See system logs and 'systemctl status httpd.service' for details.
[ec2-user@ip-172-31-33-212 ~]$ sudo systemctl start httpd
[ec2-user@ip-172-31-33-212 ~]$ systemctl status httpd
• httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; preset: disabled)
   Active: active (running) since Mon 2026-03-23 14:57:29 UTC; 10s ago
     Docs: man:httpd.service(8)
  Main PID: 31438 (httpd)
    Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0 B/sec"
   Tasks: 177 (limit: 1067)
  Memory: 13.3M
    CPU: 71ms
   CGroup: /system.slice/httpd.service
           └─31438 /usr/sbin/httpd -DFOREGROUND
             └─31439 /usr/sbin/httpd -DFOREGROUND
               └─31440 /usr/sbin/httpd -DFOREGROUND
                 └─31441 /usr/sbin/httpd -DFOREGROUND
                   └─31442 /usr/sbin/httpd -DFOREGROUND

Mar 23 14:57:28 ip-172-31-33-212.eu-north-1.compute.internal systemd[1]: Starting httpd.service - The Apache HTTP Server...
Mar 23 14:57:29 ip-172-31-33-212.eu-north-1.compute.internal systemd[1]: Started httpd.service - The Apache HTTP Server.
Mar 23 14:57:29 ip-172-31-33-212.eu-north-1.compute.internal httpd[31438]: Server configured, listening on: port 80
```

- **systemctl stop nginx** → stop service.

```
[ec2-user@ip-172-31-38-129 ~]$ sudo systemctl stop httpd
[ec2-user@ip-172-31-38-129 ~]$ systemctl status httpd
○ httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; preset: disabled)
   Active: inactive (dead)
     Docs: man:httpd.service(8)
```

7.WEB HOSTING

Web hosting in Linux means using a Linux-based server to store, manage, and deliver websites over the internet. It relies on open-source software like Apache, Nginx, PHP, MySQL, and secure tools, making it the most popular and cost-effective hosting environment worldwide.

Hands on Use Case (EC2 Linux Hosting)

1. Launch an **EC2 Linux instance**.

My Linux system	i-00b2c086c7af285be	Running	t3.micro	3/3 checks passed	View alarms +	eu-north-1b
-----------------	---------------------	---------	----------	-------------------	---------------	-------------

2. Install a web server (**sudo yum install httpd**).

```
[ec2-user@ip-172-31-38-129 ~]$ yum install httpd
Error: This command has to be run with superuser privileges
[ec2-user@ip-172-31-38-129 ~]$ sudo yum install httpd
```

3. Configure **security group rules** (open ports 80/443).

▼ Network settings [Info](#)

Network [Info](#)

vpc-08d23a3e5bc324ead

Subnet [Info](#)

No preference (Default subnet in any availability zone)

Auto-assign public IP [Info](#)

Enable

Firewall (security groups) [Info](#)

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

☒ Create security group

☐ Select existing security group

We'll create a new security group called 'launch-wizard-7' with the following rules:

☒ Allow SSH traffic from

Helps you connect to your instance

Anywhere

0.0.0.0/0

☒ Allow HTTPS traffic from the internet

To set up an endpoint, for example when creating a web server

☒ Allow HTTP traffic from the internet

To set up an endpoint, for example when creating a web server

⚠ Rules with source of 0.0.0.0/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow a known IP addresses only.

- Start httpd by **sudo systemctl start httpd**. Check the status to see its active.

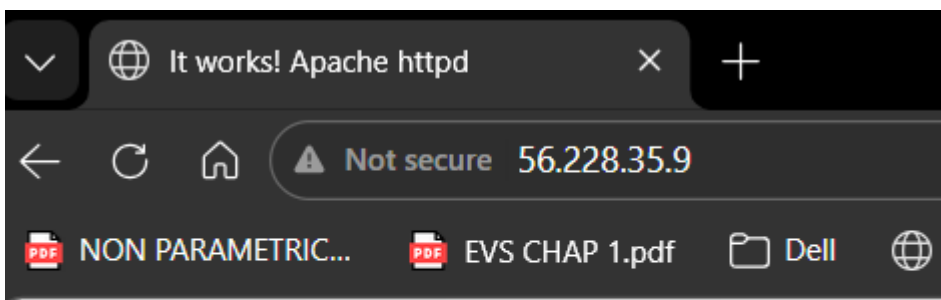
```
ec2-user@ip-172-31-38-129 ~$ sudo systemctl start httpd
ec2-user@ip-172-31-38-129 ~$ systemctl status httpd
httpd.service - The Apache HTTP Server
  Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; preset: disabled)
  Active: active (running) since Mon 2026-03-23 15:39:11 UTC; 30s ago
    Docs: man:httpd.service(8)
  Main PID: 26701 (httpd)
  Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0 B/sec"
    Tasks: 177 (limit: 1067)
  Memory: 13.4M
    CPU: 108ms
  CGroup: /system.slice/httpd.service
          └─26701 /usr/sbin/httpd -DFOREGROUND
            └─26726 /usr/sbin/httpd -DFOREGROUND
              └─26727 /usr/sbin/httpd -DFOREGROUND
                └─26728 /usr/sbin/httpd -DFOREGROUND
                  └─26729 /usr/sbin/httpd -DFOREGROUND

Mar 23 15:39:11 ip-172-31-38-129.eu-north-1.compute.internal systemd[1]: Starting httpd.service - The Apache HTTP Server.
Mar 23 15:39:11 ip-172-31-38-129.eu-north-1.compute.internal systemd[1]: Started httpd.service - The Apache HTTP Server.
Mar 23 15:39:11 ip-172-31-38-129.eu-north-1.compute.internal httpd[26701]: Server configured, listening on: port 80
```

- Access via **public IP** in an incognito tab.

i-00b2c086c7af285be (My Linux system)

PublicIPs: **56.228.35.9** PrivateIPs: 172.31.38.129



It works!